

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 04 -

IMPLANTAÇÃO DE MODELOS (DEPLOYMENT)

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
MERCADO, CASES E TENDÊNCIAS	27
O QUE VOCÊ VIU NESTA AULA?	30
REFERÊNCIAS.....	31

EMSE

O QUE VEM POR AÍ?

Na aula anterior, exploramos o desenvolvimento e a engenharia de modelos, incluindo a construção de um modelo preditivo para detecção de doença cardíaca com dados da UCI. Entretanto, o ciclo de vida de um modelo de Machine Learning não termina quando obtemos um modelo treinado no notebook. Conforme a metodologia CRISP-DM, após a fase de Avaliação vem a fase crucial de Implantação – é nela que o modelo gera valor de negócio ao ser colocado para uso real. De fato, um modelo não é particularmente útil até que os usuários finais ou sistemas da empresa possam acessar seus resultados e integrá-los em decisões ou produtos.

Implantar um modelo significa passar do ambiente de desenvolvimento para um ambiente de produção, conectando o algoritmo criado a aplicações reais utilizadas no dia a dia. Ou seja, é transformar aquele modelo treinado em um serviço ou componente que outros sistemas e pessoas possam utilizar de forma confiável e escalável. Nesta aula, vamos entender por que a implantação é importante (e desafiadora), o que envolve tecnicamente colocar modelos em produção, quem participa desse processo (papéis como engenheiros de ML, DevOps etc.), e como podemos realizar a implantação na prática usando ferramentas modernas.

É importante ter em mente que deployment não se resume a “salvar um modelo” – envolve considerações de engenharia de software, infraestrutura e operações (MLOps). Precisamos garantir que o modelo opere de forma confiável, eficiente e segura no ambiente de produção, lidando com preocupações de escalabilidade, monitoramento e manutenção contínua. Nesta aula, iremos assumir o papel de um Machine Learning Engineer que deve empacotar o modelo treinado, escolher uma estratégia de deployment adequada e integrar o modelo ao fluxo de trabalho da aplicação.

Vamos começar motivando a necessidade de deployment e então mergulhar nos aspectos técnicos. Prepare-se para pensar além do Jupyter Notebook: afinal, como fazer seu modelo chegar nas mãos de quem precisa dele?

HANDS ON

Dando continuidade ao nosso cenário aplicado à área da Saúde, temos um modelo de classificação de doença cardíaca treinado (um modelo Random Forest). Vamos explorar duas abordagens de deployment dele: via uma API REST simples com o framework Flask e usando o MLFlow Model Serving em um ambiente da Databricks. O objetivo é mostrar um fluxo de decisão entre estratégias e também pôr a mão na massa em um caso de uso real.

Na Seção Saiba Mais vamos ver o passo a passo deste modelo.

SAIBA MAIS

Parte 1: Deploy via Flask (API REST) – modelo como microserviço

Optamos por deployment em tempo real via API, pois queremos que aplicações possam obter predições de risco cardíaco sob demanda. Não vamos implantar em um app neste exemplo, pois nosso papel será disponibilizar a API de consumo do modelo para que possa ser posteriormente utilizado por outros sistemas. No conceito de microserviço, um sistema é dividido em um conjunto de serviços menores e independentes, que se comunicam através de APIs bem definidas (geralmente REST). Cada serviço é responsável por uma funcionalidade específica do negócio e pode ser desenvolvido, implantado e escalado de forma autônoma, usando diferentes tecnologias e linguagens de programação se necessário.

Fluxo de trabalho:

- **Salvar o modelo treinado** – no notebook de treinamento utilizado na aula anterior, ao final do notebook, adicionamos código para salvar o modelo final e quaisquer artefatos necessários. Optamos por usar joblib para salvar o modelo (`best_random_forest.joblib`) e assim podemos carregá-lo posteriormente para utilização na nossa API.

```
# Persistência local com joblib
models_dir = "../models"
os.makedirs(models_dir, exist_ok=True)
joblib_path = os.path.join(models_dir,
f"best_random_forest.joblib")
joblib.dump(best_rf, joblib_path)
print(f"Modelo salvo via joblib em: {joblib_path}")
```

Código-fonte 1 - Persistência local com joblib (Python)

Fonte: Elaborado pelo autor (2026)

- **Criar aplicação Flask** – desenvolveremos uma aplicação Flask `app.py` que carrega o modelo e configura dois endpoints: um raiz ("/") apenas para teste da saúde do endpoint ("health") que aceita o método GET e um `/heart-disease-predict` que aceita o método POST. Neste caso, para simplicidade, faremos o `/heart-disease-predict` (método POST) receber um payload em formato json e retornar o resultado (também em formato json).

- **Ajustar a aplicação para o Pipeline** – o ponto crucial é garantir que o microserviço Flask carregue e utilize exatamente o mesmo objeto Pipeline do sklearn (que já contém o RandomForest como último estágio) para processar as novas requisições. Esta aplicação também irá utilizar métodos de pré-processamento que irão realizar tanto os processos de OHE (One-Hot-Encoding) quanto também o processo de engenharia de features (Feature Engineering) visto em aulas anteriores. Importante: em otimizações e ajustes futuros, estas etapas poderiam também ser consideradas como parte do pipeline final produtivo em conjunto com o processo de seleção de features e encapsuladas na aplicação.

Passos Essenciais:

- **Carregamento do Pipeline:** o arquivo `best_random_forest.joblib`, que armazena o fluxo de pré-processamento com o processo de seleção de features e o modelo final (RandomForest), deve ser carregado na memória do servidor Flask na inicialização.
- **Preparação dos Dados:** os dados de entrada da requisição JSON são convertidos para um DataFrame do Pandas para manter o formato tabular que o Pipeline espera. É utilizado um módulo de pré-processamento `preprocessing.py` que aplica as transformações de OHE (One-Hot-Encoding) e o Feature Engineering.
- **Validação de Features:** o DataFrame de entrada é validado e as colunas são reordenadas para coincidir com as colunas usadas durante o treinamento do Pipeline.
- **Predição Unificada:** o DataFrame é passado diretamente para o método `.predict()` do objeto Pipeline. O Pipeline se encarrega de aplicar a pré-seleção de features antes de alimentar o modelo RandomForest para obter o resultado final.
- **Resposta:** o resultado da predição é formatado e retornado ao cliente.
- **Testar localmente:** rodamos `flask run` ou `python app.py` localmente. Inserimos alguns valores de teste (que correspondam a casos conhecidos, ou simplesmente aleatórios) e verificamos se a predição faz sentido (por exemplo, retornando "Alto Risco" para um paciente com atributos extremos).

```
~/Personal-Labs/introducao-ciclo-de-vida-de-modelos on main! .venv 13:10:08
$ export FLASK_APP=aula_04_implantacao/flask-app/app.py
python aula_04_implantacao/flask-app/app.py
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://10.248.125.110:5000
Press CTRL+C to quit
```

Figura 1 - Aplicação Flask executando localmente

Fonte: Elaborado pelo autor (2026)

```
curl -X POST http://localhost:5000/heart-disease-predict \
-H "Content-Type: application/json" \
-d '{
  "age": 63, "sex": 1, "cp": 3, "trestbps": 145, "chol":
233,
  "fbs": 1, "restecg": 0, "thalch": 150, "exang": 0,
  "oldpeak": 2.3, "slope": 0, "ca": 0, "thal": 1
}'
```

Código-fonte 2 - Exemplo de request da API

Fonte: Elaborado pelo autor (2026)

- **Container Docker:** podemos também disponibilizar a API a partir de um container Docker. Esta é uma prática comum em projetos para que possamos disponibilizar a aplicação de forma padronizada, isolada e portátil em diferentes ambientes (desenvolvimento, teste, produção), garantindo que ela funcione de maneira consistente independentemente do sistema operacional do servidor que hospedará a aplicação.
- **Produção:** em um ambiente produtivo real, colocaríamos a aplicação em um ambiente com múltiplas instâncias por trás de um load balancer, ou poderíamos utilizar um serviço orquestrador (Kubernetes, AWS EKS, Azure AKS etc.) para gerenciar e escalar automaticamente essas instâncias, garantindo alta disponibilidade e resiliência. Além disso, seria necessário implementar pipelines de CI/CD para automatizar a construção, o teste e a implantação do modelo, juntamente com sistemas de monitoramento e logging (como Prometheus, Grafana etc.) para acompanhar o desempenho e identificar problemas rapidamente.

Parte 2: Deploy via MLflow Model Serving (Databricks)

Agora, para oferecer um contraste mais “enterprise”, exploraremos como servir o mesmo modelo utilizando o framework MLFlow em uma plataforma baseada em

nuvem. Utilizaremos para isso o Databricks, que possibilitará o aproveitamento de uma arquitetura completa baseada em infraestrutura em nuvem. O Databricks é uma plataforma unificada para Data e IA (Inteligência Artificial) com as seguintes características:

- **Plataforma Unificada:** combina funcionalidades de Data Warehousing (armazenamento e análise estruturada de dados) e Data Lake (armazenamento de grandes volumes de dados brutos) em uma arquitetura chamada Lakehouse.
- **Baseada em Spark:** foi criado pelos desenvolvedores originais do Apache Spark, sendo otimizado para rodar grandes cargas de trabalho de processamento de dados (ETL, streaming) e Machine Learning, com processamento paralelo e distribuído.
- **Para o Ciclo de Vida de ML/IA:** oferece ferramentas integradas, incluindo o MLFlow, para gerenciar o desenvolvimento, o rastreamento, o deployment e o monitoramento de modelos de Machine Learning em escala (MLOps).
- **Ambiente Colaborativo:** permite que cientistas de dados, engenheiros de dados e analistas trabalhem juntos, usando linguagens como Python, R, Scala e SQL em um ambiente de notebooks interativos e clusters escaláveis.

A plataforma é amplamente utilizada em ambientes corporativos para simplificar a engenharia de dados e acelerar a jornada de Machine Learning. Nos exercícios da aula, podemos utilizar o [Databricks Free Edition](#), que permite o uso para pequenas prototipações e laboratórios de desenvolvimento, de forma gratuita.

Fluxo:

Acessar o Databricks Free Edition: o MLFlow é um framework Open Source que pode também ser utilizado localmente (como vimos nas aulas anteriores), porém para uma experiência mais completa, recomendamos logar no Databricks Free Edition conforme a documentação compartilhada anteriormente. Na plataforma é possível integrar um repositório Github (que será compartilhado como parte dos materiais), a fim de poder ter o controle de todo o processo de versionamento dos códigos, bem como posteriormente construir os pipelines de CI/CD (que serão abordados em aulas posteriores).

Treinamento de um Pipeline completo com registro no MLFlow: podemos realizar o treinamento do modelo utilizando para isso conceitos mais avançados de tratamento de dados como um Pipeline único. Podemos registrar as etapas como parte da estrutura de experimentação do MLFlow. Para isso, configuramos um experimento que chamamos de heart-disease-prediction para poder fazer tracking de todo o processo, desde o treinamento até o deployment. Neste caso, a utilização da plataforma desde o começo permitiria o acompanhamento de toda a jornada de experimentação, abrangendo desde o treinamento do modelo base (baseline) até a otimização dos hiperparâmetros, deployment etc. Também utilizamos um recurso para passarmos ao MLFlow um exemplo de assinatura do modelo; este processo auxilia na validação de entrada (input validation) e garante que o modelo seja chamado com os tipos de dados e formato corretos, facilitando a sua posterior inferência e integração em diferentes ambientes.

```
pipeline = Pipeline([
    ('imputer', MissingValueImputer(numeric_cols=numeric_cols, categorical_cols=categorical_cols)),
    ('categorical_encoding', CategoricalEncoder()),
    ('feature_engineering', FeatureEngineer()),
    ('scaler', StandardScaler()),
    ('classifier', RandomForestClassifier(**relevant_params))
])
```

Figura 2 - Exemplo de Pipeline completo com todas as etapas encadeadas (1)
Fonte: Elaborado pelo autor (2026)

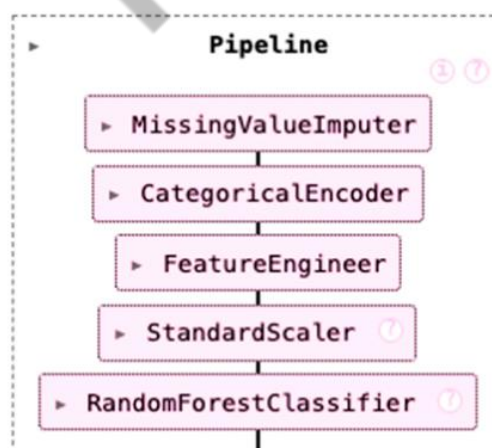


Figura 3 - Exemplo de Pipeline completo com todas as etapas encadeadas (2)
Fonte: Elaborado pelo autor (2026)

```
mlflow.set_experiment('/heart-disease-prediction')

with mlflow.start_run(run_name="complete_pipeline_with_best_params"):
    # Logar hiperparâmetros do modelo
    for param_name, param_value in relevant_params.items():
        mlflow.log_param(param_name, param_value)

    # Logar métricas
    mlflow.log_metric("train_accuracy", train_score)
    mlflow.log_metric("test_accuracy", test_score)

    # Criar signature com dados limpos (entrada do pipeline)
    signature = infer_signature(X_train, pipeline.predict(X_train))

    # Logar o pipeline completo
    model_info = mlflow.sklearn.log_model(
        sk_model=pipeline,
        artifact_path="model",
        signature=signature,
        input_example=X_train.iloc[:5]
    )

    print(f"Pipeline logged successfully!")
    print(f"Model URI: {model_info.model_uri}")
    print(f"\nHiperparâmetros logados:")
    for param_name, param_value in relevant_params.items():
        print(f"    {param_name}: {param_value}")

mlflow.end_run()
```

Figura 4 - Log do Pipeline no MLFlow
Fonte: Elaborado pelo autor (2026)

Registrar no Model Registry: usando a interface do MLFlow (ou via API), registramos o modelo treinado com um nome, como por exemplo heart-disease-model. Marcamos essa versão como "Staging" para testes e assim que tivermos certeza que o modelo possa estar pronto para produção poderíamos marcá-lo como "Production", por exemplo.

```
from mlflow import MlflowClient

# Set Unity Catalog
mlflow.set_registry_uri("databricks-uc")

# Registrar modelo
model_registered = mlflow.register_model(model_uri = mlflow_model.model_uri,
                                         name = 'heart-disease-model')

# Criar alias Staging/Production e setar ao modelo
MlflowClient().set_registered_model_alias(model_registered.name,
                                         "Production",
                                         model_registered.version)
```

> [See performance \(4\)](#)

Registered model 'heart-disease-model' already exists. Creating a new version of this model...
Created version '4' of model 'workspace.default.heart-disease-model'.

Figura 5 - Registro do modelo como Staging/Production
Fonte: Elaborado pelo autor (2026)

Servir no Databricks: no Databricks Model Serving, bastaria ir na aba do modelo e clicar em "Serve this model" para implantar o endpoint (API). Em cenários reais, poderíamos utilizar o Databricks SDK para deployment via código.

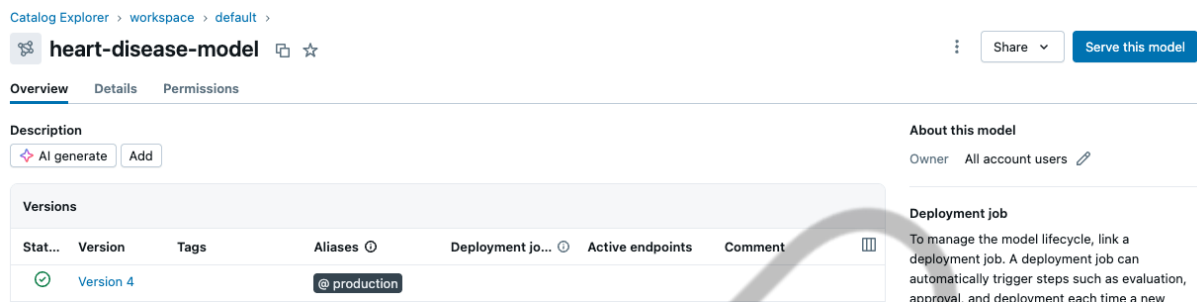


Figura 6 - MLFlow model serving
Fonte: Elaborado pelo autor (2026)

Consumir o modelo servido: após o deployment do endpoint, podemos consumir o modelo via REST API da mesma forma que vimos no exercício anterior quando construímos a aplicação Flask.



Figura 7 - API REST disponibilizada via MLFlow Serving
Fonte: Elaborado pelo autor (2026)

Quando usar uma abordagem totalmente customizada (Flask) vs quando usar uma plataforma com infraestrutura gerenciada:

- Para um cenário corporativo com muitos modelos e necessidade de governança, uma plataforma (Databricks MLflow, SageMaker, Azure ML etc.) acelera o desenvolvimento e evita reinventar rodas de deployment,

além de oferecer recursos prontos de monitoramento, autoscaling e segurança integrada (ex.: autenticação), entre outros fatores.

- Para protótipos simples ou equipes que já tenham grande domínio de administração de ambientes de microsserviços, uma aplicação com Flask container pode ser mais flexível para custom logic. Também pode ser mais barato se já temos infraestrutura disponível e alocada, porém deve-se tomar cuidado com custos indiretos relacionados ao gerenciamento de todas as dependências da solução; no final, o TCO (Total Cost of Ownership) pode não ser vantajoso.
- Plataformas baseadas em nuvem simplificam a operação de modelos em escala, oferecendo vantagens cruciais em relação a soluções self-hosted com relação à governança e rastreabilidade de experimentos, porém podem trazer vendor lock-in e dependência da infraestrutura do provedor.

A construção de uma aplicação customizada com Flask oferece alta flexibilidade e controle total sobre a arquitetura e deployment, sendo ideal para protótipos ou equipes com forte domínio de DevOps e gerenciamento de aplicações baseadas em microsserviços. Contudo, exige grande esforço de engenharia para construir e manter ferramentas de MLOps (rastreamento, registro e monitoramento), elevando a complexidade e os custos indiretos de manutenção, além de exigir trabalho para escalabilidade.

Por outro lado, plataformas gerenciadas como Databricks MLflow são menos flexíveis, mas oferecem governança e MLOps de forma nativa (rastreamento, registro e segurança), abstraindo a complexidade da infraestrutura. São altamente escaláveis e operacionais, sendo a escolha mais eficiente para ambientes corporativos com muitos modelos e onde o time-to-market é prioridade, apesar de terem um custo direto mais elevado pelo serviço gerenciado e potencialmente maiores dificuldades com vendor lock-in.

Ao final do hands on, podemos ver duas abordagens: uma low-level (construir seu próprio serviço) e outra high-level (usar um serviço gerenciado). Escolher a melhor abordagem dependerá do contexto de cada cenário, considerando fatores como volume de dados, especialização dos times, requisitos de escalabilidade e infraestrutura disponível.

Conceitos e Importância do Deployment de Modelos

Implantar um modelo de ML é o processo de colocar em operação um modelo treinado de forma que ele possa receber dados novos e produzir previsões para usuários ou outros sistemas. Em outras palavras, é encaixar o modelo dentro de uma aplicação maior. Essa etapa é fundamental porque é só após o deployment que o modelo começa, de fato, a gerar valor para a organização – seja automatizando decisões, suportando usuários com insights ou habilitando novos produtos inteligentes.

Do ponto de vista de negócio, um projeto de Data Science ou ML só cumpre seu propósito quando seus resultados são integrados ao processo de negócio ou à experiência do usuário final. Isso pode significar embutir um modelo de recomendação em um e-commerce (para sugerir produtos em tempo real), usar um modelo de previsão de risco em um sistema bancário (para aprovar ou negar transações suspeitas instantaneamente) ou rodar um modelo de manutenção preditiva periodicamente na base de equipamentos de uma fábrica (gerando relatórios semanais). Sem implantação, mesmo o modelo mais preciso permanece como uma prova de conceito isolada.

Além do ganho de valor, o deployment também traz novos desafios e responsabilidades. Diferentemente do ambiente controlado de um notebook, em produção o modelo precisará lidar com:

- **Dados de entrada imprevisíveis ou diferentes do treino** (exigindo robustez e possível retreinamento se houver drift).
- **Exigências de desempenho e escala** – por exemplo, atender centenas de requisições por segundo com baixa latência ou processar em lote milhões de registros dentro de janelas de tempo definidas.
- **Integração com sistemas existentes** (bancos de dados, sistemas legados, interfaces de usuário).
- **Confiabilidade, observabilidade e manutenção contínua** – o modelo em produção precisa ser monitorado (performance, erros, acurácia ao longo do tempo) e deve haver planos de contingência caso o serviço de previsão falhe.

- **Segurança e privacidade** – proteger dados sensíveis durante a inferência, controlar acesso ao serviço do modelo, prevenir usos indevidos ou exposições do modelo (Model Card e conformidade ética, se aplicável).

Resumindo, o foco após o treinamento desloca-se para garantir que o modelo opere 24/7 de forma robusta, eficiente e segura no ambiente de produção, atendendo aos requisitos de negócio e técnicos. Isso exige uma mentalidade de engenharia e operacionalização (OPS): pensamos em implantação, como software, com versionamento, testes, deploy automatizado, monitoramento e protocolos de rollback se necessário – é o domínio do MLOps (Machine Learning Operations).

Sem deployment, o trabalho de Data Science fica confinado a reports ou notebooks. Trazer o modelo para produção permite ações automatizadas em escala. Por exemplo, em vez de um(a) analista olhar um relatório e manualmente tomar decisões, um modelo implantado pode aprovar/reprovar automaticamente um pedido de empréstimo, detectar e bloquear uma transação fraudulenta em tempo real ou personalizar a experiência de milhares de usuários em um aplicativo. Isso aumenta a velocidade e o alcance das decisões baseadas em ML no negócio. Além disso, a implantação contínua de modelos faz parte de uma cultura orientada a dados, onde modelos passam por ciclos de melhorias constantes, incorporando novos dados e feedbacks.

Do ponto de vista técnico, podemos destacar os seguintes desafios: (a) Desafio de Engenharia: integrar o modelo em uma aplicação de produção nem sempre é trivial – pode ser necessário desenvolver APIs, pipelines de dados e lidar com diferenças de ambientes; (b) Escalabilidade: um modelo que funciona bem com poucos usuários pode enfrentar problemas sob carga real. É preciso projetar a arquitetura para escalar horizontalmente (mais instâncias) ou verticalmente (mais recursos) conforme a demanda, garantindo tempo de resposta adequado mesmo nos picos; (c) Segurança: ao expor um modelo como serviço, precisamos considerar controle de acesso e criptografia de dados sensíveis em trânsito e em repouso, além de estar em conformidade com regulamentações (por exemplo, GDPR, LGPD no caso de dados pessoais). Modelos muitas vezes lidam com dados de clientes, logo privacidade e segurança são cruciais – técnicas como mascaramento de dados, autenticação em APIs e monitoração de abusos (para evitar extração do modelo via consultas, por exemplo) devem ser consideradas.

Outros desafios incluem manutenção e monitoramento (como detectar drift de dados ou degradação da acurácia do modelo ao longo do tempo e saber quando atualizar ou retreinar) e balancear velocidade vs. custo (por exemplo, servir um modelo com latência baixíssima pode exigir mais recursos computacionais, aumentando custos – é preciso achar um equilíbrio conforme o caso de uso).

Em suma, implantar modelos é tão importante quanto desenvolvê-los, porém traz um novo conjunto de preocupações.

Estratégias de Implantação de Modelos

Não existe uma única forma de colocar um modelo em produção. A estratégia ideal depende de vários fatores, como: urgência das previsões (tempo real vs batch), volume de dados/predições necessários, ambiente onde o modelo será executado (nuvem, dispositivos locais, ambiente web, mobile etc.) e como os resultados serão consumidos (por um usuário final via interface? por outro sistema via API? armazenados em lote em um banco de dados?). Vamos explorar as principais modalidades:

- **Implantação Batch** – o modelo é executado periodicamente em blocos de dados, com atualização pontuais (diárias, semanais, mensais etc.).
- **Implantação em Tempo Real (online)** – o modelo é servido como um serviço de consulta de baixa latência e consumido sob demanda.
- **Implantação na Borda (Edge/On-device)** – o modelo é incorporado dentro de uma aplicação ou dispositivo, rodando localmente.
- **Arquiteturas de Streaming assíncrono** – o modelo processa fluxo contínuo de eventos com baixa latência, tipicamente via filas.

Vamos detalhar cada uma, comparando vantagens e casos de uso típicos.

Predições em Batch (Processamento em Lote)

Na implantação batch, as predições do modelo são realizadas em blocos, seguindo uma cadência programada ou conforme demanda manual. Ou seja, em vez de avaliar um caso por vez sob demanda, acumulam-se vários inputs e o modelo os processa todos juntos periodicamente (por exemplo, diariamente, semanalmente ou em intervalos fixos).

Quando usar: casos em que não é necessário obter o resultado imediatamente para cada dado individual. Exemplos clássicos:

- Geração de relatórios preditivos periódicos (ex.: risco de churn de clientes calculado todo primeiro dia do mês).
- Recalcular recomendações de produtos para todos os usuários durante a madrugada, para alimentar a homepage de um e-commerce no dia seguinte.
- Processar lote de solicitações acumuladas, como por exemplo análises de crédito pré-processadas overnight.

As vantagens incluem eficiência e simplicidade. Como os jobs podem rodar em horários de baixo uso (off-peak) ou em segundo plano, otimiza-se o uso de recursos e pode-se aproveitar máquinas mais potentes para grandes volumes de uma vez. O código de deployment em batch muitas vezes pode ser implementado como um script simples que carrega o modelo e processa um arquivo ou tabela inteira – sem necessidade de lidar com servidores web ou altos requisitos de disponibilidade. Também é mais fácil de testar e reproduzir (execução determinística em dados conhecidos).

Sobre as desvantagens, há latência entre a disponibilidade dos dados e a obtenção das predições. Se algo relevante muda após a última execução, o modelo só refletirá isso no próximo agendamento. Portanto, não serve para aplicações que precisam de resposta instantânea ou em tempo real. Além disso, mesmo em batch, devemos nos preocupar com versionamento de modelo e ambiente – diferentes jobs devem usar a versão correta do modelo (dica: usar pipeline automatizado com data+versão do modelo).

Implementação típica: usar um job scheduler ou orquestrador (como Apache Airflow, Databricks Jobs, etc.) que em intervalos definidos executa um script. Esse script carrega o modelo treinado (geralmente de um arquivo ou de um registro de modelos), lê um lote de dados de fonte (um arquivo CSV, uma tabela em data warehouse etc.), aplica `model.predict()` em todos e escreve os resultados em algum destino (banco, sistema de arquivos, mensagem). Ferramentas de big data como Spark também podem ser utilizadas para escalar o processamento em lote, distribuindo as predições.

No nosso exemplo de modelo de doença cardíaca, poderíamos imaginar uma implantação batch onde, toda noite, um script extrai do banco de dados hospitalar os pacientes novos cadastrados e roda o modelo nesses registros, gerando uma lista de pacientes com alto risco que é enviada aos médicos pela manhã. Esse processamento não precisa ser instantâneo após cada cadastro, podendo ocorrer em lote diário.

Predições em Tempo Real (Serviço Online)

Na implantação em tempo real, o modelo fica disponível como um serviço que responde imediatamente a requisições individuais, tipicamente através de uma API REST. É como se o modelo fosse “colocado atrás de um endpoint” no qual um cliente pode fazer uma requisição (por exemplo, enviando as features) e receber a predição resultante quase instantaneamente (em poucos milissegundos ou segundos).

Quando usar: aplicações em que a inferência precisa acontecer imediatamente para cada entrada ou evento, geralmente porque há um usuário ou sistema aguardando a resposta para tomar uma ação. Exemplos:

- Um usuário preenche um formulário online pedindo um empréstimo e o aplicativo chama um modelo de credit scoring para aprovar ou negar em tempo real antes de finalizar a requisição.
- Um sistema de detecção de fraude analisa cada transação bancária no momento em que ela acontece e decide bloqueá-la ou não instantaneamente.
- Um chatbot ou assistente virtual que responde com ML (por ex, modelo de NLP) às perguntas do usuário enquanto ele interage, exigindo respostas em segundos.

A principal vantagem é experiência em tempo real – mínima latência entre o input e a decisão. Isso viabiliza produtos interativos e decisões automáticas imediatas, aumentando a utilidade do modelo em cenários dinâmicos. Além disso, modelos servidos via API podem ser facilmente integrados a múltiplos sistemas e interfaces (web, mobile, outros serviços) de forma padronizada.

Sobre desafios, ele requer infraestrutura de serving sempre disponível e capaz de lidar com concorrência. Em vez de um script que roda e termina, aqui geralmente precisamos manter um servidor (por exemplo, um processo Flask/FastAPI ou um

servidor de modelos) ativo 24/7. Esse servidor carrega o modelo na memória e aguarda requisições. É crucial garantir baixa latência – o sistema deve estar otimizado para responder rápido, possivelmente fazendo caching de modelo, usando hardware apropriado (CPU vs GPU) e escalando para múltiplas instâncias se o número de requisições por segundo for alto. Também entra em jogo a tolerância a falhas: precisamos monitorar se o serviço do modelo está no ar e talvez ter instâncias redundantes para alta disponibilidade.

Outro ponto é que em tempo real geralmente lidamos com uma entrada por vez, o que pode ser menos eficiente computacionalmente do que processar em lote. Ferramentas de serving modernos tentam amenizar isso (por ex, fazendo micro-batching interno), mas é algo a considerar – servir 1000 requisições de uma em uma costuma ter overhead maior do que processar as 1000 de uma vez em batch. Contudo, muitas aplicações não têm escolha devido ao requisito de latência por requisição.

APIs REST vs. outras interfaces: vale notar que REST APIs são a estratégia mais comum para tempo real pela simplicidade e ubiquidade (qualquer linguagem e plataforma consegue consumir uma API HTTP). Em alguns casos de latência ultra baixa, pode-se usar gRPC (que é binário e mais performático) ou até implementar o modelo como uma função dentro do próprio aplicativo consumidor (embedded no código do backend, se estiver na mesma stack). Mas esse último tende a dificultar a manutenção, por isso separar em um serviço via API tem vantagens de isolamento e escalabilidade.

Modelos Embarcados em Aplicações (Edge AI)

Outra estratégia é embutir o modelo diretamente em uma aplicação ou dispositivo, em vez de servir via requisição a um servidor remoto. Este cenário é comum quando precisamos que a inferência ocorra localmente, seja por requerimento de latência ultra baixa, limitação de conectividade ou considerações de privacidade. Chamamos de edge deployment quando o modelo roda “na ponta” – pode ser dentro do aplicativo mobile do usuário, no navegador web, em um dispositivo IoT ou em um sistema instalado em uma máquina local.

Quando usar:

- **Aplicações mobile ou de dispositivos inteligentes:** por exemplo, um app de celular com um modelo de reconhecimento de imagem integrado (como modelos de câmera que detectam objetos, rosto do usuário para desbloqueio etc.), rodando no próprio smartphone sem precisar de internet.
- **Cenários com conexão instável/inexistente ou latência crítica:** drones ou carros autônomos que não podem depender de nuvem para decisões instantâneas; sensores em fábricas ou em campo que executam modelos localmente para detectar anomalias e só enviam os resultados.
- **Privacidade/segurança:** quando os dados não podem (ou não se quer) enviar para um servidor. Ex.: um modelo que processa dados médicos ou informações do usuário diretamente no dispositivo do usuário, mantendo os dados localmente.
- **Integração em software local:** algumas aplicações desktop podem incluir um modelo treinado para oferecer alguma funcionalidade inteligente sem requerer conexão (por exemplo, um software de edição de fotos trazendo um modelo de filtro de imagem embutido).

A inferência pode ser muito rápida (pois elimina o tempo de rede) e funciona mesmo offline. Além disso, ao manter dados localmente, reduz-se a superfície de ataque e problemas de compliance – não há transmissão de dados sensíveis, o que minimiza riscos de privacidade e vazamento. A experiência do usuário pode melhorar (um exemplo: recurso de autocompletar do teclado do celular que funciona mesmo sem internet, pois o modelo de linguagem está no aparelho). Também pode reduzir custos de infraestrutura em nuvem, já que parte do processamento é distribuído nos dispositivos dos usuários.

No entanto, é um desafio quando o modelo exige muito recurso computacional – embora dispositivos móveis e edge estejam ficando poderosos, ainda há limites de CPU/GPU e especialmente de energia (bateria). Muitas vezes é preciso otimizar o modelo para caber e rodar nesses ambientes. Outra desvantagem é a distribuição e atualização: uma vez que o modelo está embarcado no app ou device, para atualizá-lo geralmente é necessário atualizar a versão do aplicativo ou enviar firmware novo, o que pode ser menos ágil do que atualizar um modelo no servidor. Além disso, perde-

se um pouco de controle – se um bug aparece no modelo em edge, não dá para "desligar" facilmente, diferente de um serviço centralizado. Monitorar a performance do modelo também é mais difícil, pois você não vê diretamente as previsões sendo feitas (pode-se mitigar coletando telemetria anônima, por exemplo).

Implementação típica: envolve exportar o modelo para um formato executável no ambiente alvo. Aqui entram formatos como ONNX, CoreML (para iOS), TensorFlow Lite ou até compilar o modelo para C++ ou WebAssembly, dependendo do caso. Por exemplo, podemos pegar um modelo treinado em Python, exportar para ONNX e dentro de um app mobile utilizar ONNX Runtime para executá-lo. Ou no caso de um modelo simples, incorporar a lógica manualmente.

Um exemplo concreto relacionado ao nosso caso: imagine que queremos que médicos(as) em áreas remotas, sem internet, possam usar o modelo de previsão de doença cardíaca em um aplicativo de smartphone. Poderíamos exportar nosso modelo para ONNX. O app mobile incluiria o arquivo .onnx e usaria a biblioteca ONNX Runtime móvel para carregar esse modelo e fazer previsões localmente quando o indivíduo preencher os dados de um paciente no app. Assim, nenhuma informação sai do dispositivo e o resultado aparece instantaneamente, mesmo no meio da floresta amazônica. A contrapartida é que teríamos que atualizar o app para distribuir melhorias no modelo no futuro.

Streaming de Dados e Inferência Assíncrona

Na abordagem com inferência assíncrona em streaming, o modelo processa eventos em quase tempo real, mas sem interação direta com um usuário – em vez disso utiliza uma arquitetura de mensageria (filas, streams tipo Kafka). Esse padrão é útil quando se tem fluxos contínuos de dados chegando em alta velocidade e deseja-se que o modelo marque cada evento, possivelmente com uma leve defasagem.

Exemplo: detecção de fraudes ou anomalias em transações em um fluxo de eventos financeiros. As transações são colocadas em um tópico Kafka; um ou mais consumidores executam o modelo conforme os eventos chegam, colocando resultados em uma fila de saída ou banco. Assim, não há um request específico de um cliente para cada transação – o sistema vai processando continuamente. Esse método requer um sistema de fila robusto e normalmente um componente de consumidor rodando continuamente o modelo.

Pode ser considerado uma variante do tempo real, mas escalada para throughput, adicionando uma fila para bufferizar e distribuir as mensagens. A complexidade aqui aumenta (pois temos que gerenciar a fila, consumer groups, ordenação etc.), mas permite altíssimo throughput sem sobrecarregar o serviço – e também naturalmente desacopla produção e consumo (se o modelo cair, os eventos ficam na fila e podem ser processados ao retornar, garantindo nenhuma perda, embora com atraso). Ferramentas como Apache Kafka, AWS Kinesis, RabbitMQ e frameworks de processamento de streams (Flink, Spark Streaming) se encaixam nesses cenários.

Empacotamento e Portabilidade de Modelos

Uma etapa essencial antes de colocar um modelo em produção é empacotá-lo de forma que possa ser carregado e executado facilmente em outro ambiente (que não o seu ambiente de desenvolvimento). Existem diversos formatos e ferramentas para serializar modelos. Vamos discutir as principais opções e boas práticas:

- **Pickle/Joblib (serialização Python):** formas de salvar objetos Python em disco.
- **ONNX (Open Neural Network Exchange):** formato aberto para interoperabilidade entre diferentes frameworks e linguagens.
- **Outros formatos específicos:** HDF5/SavedModel (Keras/TensorFlow), PMML, entre outros.

Salvando modelos com Pickle ou Joblib

No ecossistema Python, a maneira mais direta de salvar um modelo treinado é usando o módulo padrão pickle ou a biblioteca joblib. Ambos permitem serializar objetos Python (incluindo modelos do scikit-learn, XGBoost etc.) em um arquivo binário e posteriormente recarregá-los em outro processo Python reproduzindo o estado original do objeto.

Cuidados importantes: ao usar pickle/joblib, o modelo carregado depende das mesmas versões das bibliotecas usadas no treino. Um modelo pickle de scikit-learn 1.0 pode falhar ao carregar em um ambiente com scikit-learn 1.2 se houver mudanças

incompatíveis. Portanto, é fundamental alinhar versões no ambiente de deploy (podemos usar um requirements.txt ou ambiente conda para replicar).

Tamanho do arquivo: modelos grandes (ex.: ensembles com centenas de árvores ou redes neurais enormes) podem gerar arquivos pickles grandes (centenas de MB). Isso impacta o tempo de transferência e carregamento. De qualquer forma, faz parte do pipeline garantir que esse artefato do modelo esteja acessível ao ambiente de produção (geralmente armazenando em um storage acessível ou incorporando no container Docker).

Exportação para ONNX (e outros formatos portáteis)

ONNX (Open Neural Network Exchange) é um formato aberto e padronizado para representar modelos de aprendizado de máquina de forma independente de plataforma ou linguagem. A ideia do ONNX é servir como "linguagem comum" – você treina o modelo em um framework (digamos PyTorch ou scikit-learn), exporta para ONNX e então pode carregar esse modelo ONNX para fazer inferência em diversas plataformas: em C++, em Java, em .NET, em mobile etc., usando runtimes ONNX disponíveis.

Outros formatos:

- **HDF5/SavedModel/Keras (Keras/TensorFlow):** para redes neurais com os frameworks Keras/TensorFlow, a prática comum é salvar no formato "SavedModel" ou no formato "keras", que depois pode ser servido via TF Serving ou carregado em TF Lite.
- **TorchScript:** no PyTorch, você pode compilar o modelo em um script/autônomo que também independe do Python em tempo de execução (útil para C++ deployments).

Ferramentas de Deployment de Modelos – MLflow e Model Registry

Implantar modelos manualmente (escrevendo scripts de pickle, criando servidores Flask na mão) funciona, mas em ambientes profissionais e escaláveis, contamos com ferramentas que padronizam e automatizam boa parte do processo de deployment. Uma das mais utilizadas atualmente é o MLflow, uma plataforma de ciclo de vida de ML de código aberto.

O MLflow Model Registry é, como o nome sugere, um repositório central para modelos treinados, acoplado a um sistema de versionamento. Com ele, podemos registrar cada modelo treinado com um nome e, a cada nova versão registrada, promover modelos a staging, production ou archived, adicionando comentários, tags etc. Em outras palavras, o registry traz um controle de linhagem e ciclo de vida: sabemos qual experimento/run originou o modelo, temos anotações de quem aprovou passar para produção e podemos reverter para versões anteriores facilmente.

As vantagens são muitas:

- **Organização:** não precisamos gerenciar arquivos pickle soltos com nomes confusos. Todos os modelos ficam catalogados com versionamento claro.
- **Rastreabilidade:** cada modelo registrado tem seus metadados – por exemplo, "Modelo X versão 3 – auc = 0.85, treinado com dataset Y, parâmetros Z". Sabemos exatamente qual modelo está em produção e com quais características.
- **Transições controladas:** podemos definir um fluxo: quando um modelo novo é treinado, marcamos como staging para testes; depois de avaliado, alguém aprova e muda para production e o registry mantém apenas um modelo (ou um conjunto limitado) com status production. Isso evita confusão de múltiplos modelos concorrentes.
- **Integração com deployment:** muitos frameworks (incluindo o próprio MLflow Serving ou sistemas como SageMaker, Azure ML) podem consumir modelos diretamente do registry pela versão ou estágio, facilitando automatização de deploy contínuo. Ex.: você pode rodar um script de CI/CD que diz "use o modelo X na versão marcada como Production".

Em resumo, o Model Registry nos dá governança sobre os artefatos de modelo – essencial quando múltiplos modelos e versões são gerenciados em paralelo em uma empresa.

Além de gerenciar modelos, o MLflow oferece ferramentas para deployment propriamente dito, seja local ou em diversos ambientes. O MLflow Models define um formato padrão para empacotar modelos e o MLflow fornece comandos para servir esses modelos rapidamente como serviços.

Algumas funcionalidades notáveis:

- Você pode rodar um comando `mlflow models serve -m models:/MeuModelo/Production` e o MLflow faz o deployment de um servidor REST expondo aquele modelo. Isso facilita testes rápidos ou pequenos deployments sem escrever código Flask manualmente.
- O MLflow integra com serviços externos: há comandos para deploy em AWS SageMaker, Azure ML etc, que criam containers com o modelo e os publicam nesses serviços. Isso poupa tempo de escrever integrações específicas.
- No Databricks (e recentemente outros clouds), existe o Databricks Model Serving, um serviço gerenciado onde, ao clicar para servir um modelo registrado, ele automaticamente disponibiliza um endpoint HTTPS escalável.
- Empacotamento em container: O MLflow pode gerar imagens Docker automaticamente a partir de um modelo registrado. Ele coloca o modelo em uma pequena aplicação REST com suas dependências; assim, você pode por exemplo usar essa imagem e implantar em qualquer cluster Kubernetes ou serviço de containers.

Até aqui discutimos formas de disponibilizar o modelo, mas não podemos esquecer de aspectos práticos de implantação em ambientes de TI reais: containerização e uso de infraestruturas de cloud são quase onipresentes hoje. Vamos entender brevemente como Docker e os provedores de nuvem se encaixam no ciclo de deployment.

Containerização com Docker

O Docker é uma plataforma de contêineres que permite empacotar uma aplicação e seu ambiente em uma unidade padronizada. No contexto de ML, “containerizar” um serviço de modelo significa encapsular: o código do serviço (por exemplo, nossa API Flask ou MLflow server), o arquivo do modelo e todas as dependências (bibliotecas, especificações de sistema) dentro de uma imagem Docker. Essa imagem pode então ser executada em qualquer servidor ou serviço compatível

com Docker, garantindo que o modelo rode de forma consistente em qualquer ambiente, independentemente de configuração de SO ou versões de pacotes.

Em outras palavras, se o modelo funcionou dentro do container na minha máquina, funcionará igual em produção, pois o container já inclui exatamente tudo que ele precisa (Python versão X, scikit-learn versão Y etc.). Isso elimina muitos problemas de incompatibilidade de ambiente e o “funciona na minha máquina”.

Além disso, o Docker facilita escalabilidade: podemos rodar múltiplas instâncias do mesmo container atrás de um balanceador de carga para lidar com mais tráfego. Também facilita a integração com pipelines CI/CD – podemos automatizar a construção de uma nova imagem assim que um modelo novo é aprovado e implantar essa imagem em um cluster.

O uso de Docker se tornou tão comum que ferramentas de ML geralmente já o incorporam – por exemplo, o MLflow, como dito, pode gerar Dockerfiles automaticamente e serviços de nuvem de ML (como AWS SageMaker e Azure ML) executam inferência dentro de containers padronizados.

Integrar deployment de modelos com processos DevOps significa automatizar desde o treinamento até a entrega. Um possível fluxo: um modelo é treinado e registrado (com MLflow); um pipeline CI/CD detecta um modelo marcado "Production", então dispara a construção de uma imagem Docker com aquele modelo e código de inferência; essa imagem é implantada (talvez em um cluster Kubernetes ou serviço serverless); testes de saúde são realizados; e então o novo modelo entra em uso. Ferramentas como Jenkins, GitLab CI, Azure DevOps podem orquestrá-lo. Isso traz reprodutibilidade e rapidez – novos modelos podem ir para produção sem intervenção manual pesada, porém com controle e possibilidade de rollback.

E claro, a área de MLOps está evoluindo rápido – novas ferramentas e melhores práticas surgem constantemente. A área de MLOps (Machine Learning Operations) está, sem dúvida, evoluindo a um ritmo acelerado. Novas ferramentas e melhores práticas de engenharia de software e aprendizado de máquina surgem constantemente, tornando o panorama da implantação de modelos (deployment) cada vez mais sofisticado e eficiente.

Essa rápida evolução é impulsionada pela necessidade de:

- **Acelerar a Produção:** diminuir o tempo entre o desenvolvimento de um modelo e sua entrega em um ambiente de produção que gere valor de negócio.
- **Garantir a Estabilidade:** assegurar que os modelos implantados sejam robustos, escaláveis e confiáveis, minimizando falhas e downtime.
- **Melhorar a Governança e Conformidade:** estabelecer trilhas de auditoria claras, gerenciar versões de forma rigorosa e garantir o cumprimento de regulamentos (como privacidade de dados).
- **Otimizar o Monitoramento Contínuo:** desenvolver métodos mais inteligentes para detectar drift de dados e drift de conceito, que degradam o desempenho do modelo ao longo do tempo.

Para profissionais e equipes que trabalham com a implantação de modelos, é fundamental manter-se atualizado(a) com essas tendências para garantir que os sistemas de ML permaneçam competitivos, eficientes e em conformidade com os padrões da indústria.

MERCADO, CASES E TENDÊNCIAS

Para conectar com o mundo real, vejamos como diferentes indústrias aplicam deployment de modelos:

- **E-commerce (Recomendações):** empresas como Amazon e Netflix utilizam extensivamente batch inference e online inference combinados. Por exemplo, a Netflix gera diariamente, em batch, uma lista personalizada de recomendações para cada usuário off-line (tarefa pesada processando todo o catálogo e preferências) – essas listas populam a página principal do usuário. Mas também usa tempo real para pequenas sugestões (por ex., “quem viu isso também viu aquilo” quando você clica em um filme). Esse caso ilustra bem o uso híbrido: batch para escala e abrangência, real-time para interação imediata. O deployment aqui envolve pipelines big data (Spark jobs) e microsserviços de recomendação. Tudo orquestrado para que o usuário nem perceba que modelos diferentes estão atuando.
- **Finanças (Fraude e Risco de Crédito):** bancos e empresas de cartão de crédito precisam de decisões instantâneas. Transações suspeitas são classificadas por modelos em alguns milissegundos antes de serem aprovadas – isso exige deployment em tempo real e com altíssima disponibilidade (não pode falhar e impedir compras legítimas). Tipicamente, esses modelos são implantados em ambientes distribuídos, às vezes até mais próximos dos data centers regionais para reduzir a latência. O desafio de segurança é enorme – garantir que o modelo não seja corrompido (imagine um hacker tentando forçar o modelo a sempre aprovar). Os bancos também fazem monitoramento constante: se a performance cair, há protocolos para retirar o modelo de produção.
- **Saúde (Diagnóstico Assistido):** em hospitais, cada vez mais vemos modelos auxiliando médicos, por exemplo detectando nódulos em imagens de raio-X. Muitos desses modelos inicialmente funcionam como batch – o radiologista utiliza um sistema que envia um lote de imagens ao fim do dia para um servidor que usa o modelo para destacar possíveis achados. Porém, há casos caminhando para previsão em tempo real no próprio

aparelho (edge): aparelhos de raio-X e tomografia incorporando modelos que logo após o exame já dão alertas (ex.: "suspeita de fratura detectada"). Isso mostra a tendência de edge AI em saúde para agilizar fluxos críticos. Empresas como Siemens Healthineers e GE desenvolvem esses modelos integrados aos equipamentos (a implantação aqui é embarcada em máquinas especializadas, com hardware GPU local). Uma preocupação forte é validação e certificação – modelos médicos em produção precisam passar por regulação (ex.: FDA nos EUA), o que torna o ciclo de atualização mais lento e complexo.

- **Manufatura (Manutenção preditiva):** em fábricas, sensores IoT coletam vibração, temperatura de máquinas etc. Um modelo de ML pode prever falhas antes que ocorram. O deployment típico: um dispositivo industrial (edge gateway) roda o modelo localmente para cada máquina, enviando alertas ao sistema central quando uma anomalia é detectada. Isso porque nem sempre há internet rápida no chão de fábrica ou, mesmo que haja, a latência local evita danos (se uma vibração anômala aparece, o modelo local pode acionar parada da máquina em segundos, não dependente de nuvem). Empresas adotam essa abordagem para economizar custos de parada não planejada. Um case famoso: a General Electric com sistemas Predix, que colocam modelos em turbinas de avião, locomotivas etc., operando em edge pois a conectividade em tempo de voo não permite mandar todos os dados para terra.
- **Tecnologia (Serviços web escaláveis):** em empresas tech (Google, Facebook, etc.), o deployment de modelos atinge outro nível: sistemas de recomendação de feed, classificação de conteúdo, algoritmos de busca – todos rodando em tempo real sobre bilhões de dados. Aqui se utiliza infra altamente sofisticada: modelos são deployados em serviços distribuídos na memória, muitas vezes com hardware custom (Google TPU, por exemplo). Eles empregam canary releases constantemente – cada pequena mudança de modelo é testada em 1% dos usuários, medindo a melhora antes de expandir para todos os usuários. Também fazem rollback automáticos caso as métricas caiam. Ferramentas internas gerenciam milhares de modelos simultaneamente. Ou seja, é MLOps no máximo grau. Um case

interessante: o Uber Michelangelo – plataforma interna que gerencia desde treino até deploy em escala, suportando casos de uso como estimativa de tempo de chegada, preços dinâmicos etc., sem que cada engenheiro precise reinventar o pipeline. O Uber acabou publicando artigos sobre este tema inspirando o surgimento de ferramentas open-source.

Esses exemplos reforçam que diferentes contextos pedem estratégias distintas: não existe um modelo de deployment único que sirva para tudo. O(a) profissional de Machine Learning Engineering deve avaliar requisitos e restrições do domínio para arquitetar a solução apropriada.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, cobrimos o ciclo final do desenvolvimento de modelos de ML: a Implantação em produção. Entendemos o significado de deploy de modelos e por que é uma etapa crítica para gerar valor real nos projetos de ML – sem deployment, modelos ficam no ambiente de desenvolvimento sem impacto no negócio. Discutimos a importância de pensar além do notebook, englobando requisitos de engenharia de software, infraestrutura e operações (MLOps) para manter modelos funcionando adequadamente para os usuários.

Além disso, exploramos as diferentes estratégias de deployment, discutimos empacotamento de modelos para deployment, conhecemos ferramentas e frameworks que auxiliam no deployment, destacamos a importância de Docker e containerização e abordamos desafios de engenharia, escalabilidade e segurança.

REFERÊNCIAS

AWS BLOGS. **Batch vs Real-Time Inference in AWS Machine Learning**. 2020. Disponível em: <https://aws.amazon.com/blogs>. Acesso em: 28 jan. 2026.

DATABRICKS DOCUMENTATION. **Mosaic ML Model Serving**. 2025. Disponível em: <https://docs.databricks.com>. Acesso em: 28 jan. 2026.

DATASCIENCE-PM. **What is CRISP DM?** 2024. Disponível em: <https://www.datascience-pm.com/crisp-dm-2>. Acesso em: 28 jan. 2026.

EMMANUEL, C. **Predicting Heart Disease Flask – A Practical Deployment Guide**. 2025. Disponível em: <https://chris-emmanuel.medium.com/predicting-heart-disease-with-machine-learning-and-flask-a-practical-deployment-guide-39ba5a388bf5>. Acesso em: 28 jan. 2026.

HAPKE, H.; NELSON, C. **Building Machine Learning Pipelines**. 2020. Sebastopol: O'Reilly.

MARTIN, M. **Model Deployment: Types, Strategies and Best Practices**. 2022. Disponível em: <https://dagshub.com/blog/model-deployment-types-strategies-and-best-practices>. Acesso em: 28 jan. 2026.

MEHREEN, K. **Step-by-Step Guide to Deploying ML Models with Docker**. 2024. Disponível em: <https://www.kdnuggets.com/step-by-step-guide-to-deploying-ml-models-with-docker>. Acesso em: 28 jan. 2026.

MIRANTIS. **Model Deployment and Orchestration: The Definitive Guide**. 2021. Disponível em: <https://www.mirantis.com/blog/model-deployment-and-orchestration-the-definitive-guide/>. Acesso em: 28 jan. 2026.

MLFLOW DOCUMENTATION. **MLflow Model Registry**. 2025. Disponível em: <https://mlflow.org>. Acesso em: 28 jan. 2026.

MLFLOW DOCUMENTATION. **MLflow Model Serving**. 2025. Disponível em: <https://mlflow.org>. Acesso em: 28 jan. 2026.

SCIKIT-LEARN DOCUMENTATION. **Model Persistence**. 2023. Disponível em: <https://scikit-learn.org>. Acesso em: 28 jan. 2026.

SHELF.IO. **In-depth Guide to ML Model Deployment**. 2023. Disponível em: <https://shelf.io/blog/machine-learning-deployment>. Acesso em: 28 jan. 2026.

STACKADEMIC BLOG. **MLOps: Model Deployment Strategies — Batch vs. Real-Time vs. Edge**. 2024. Disponível em: <https://blog.stackademic.com/mlops-model-deployment-strategies-batch-vs-real-time-vs-edge-c7b1ac642e8f>. Acesso em: 28 jan. 2026.

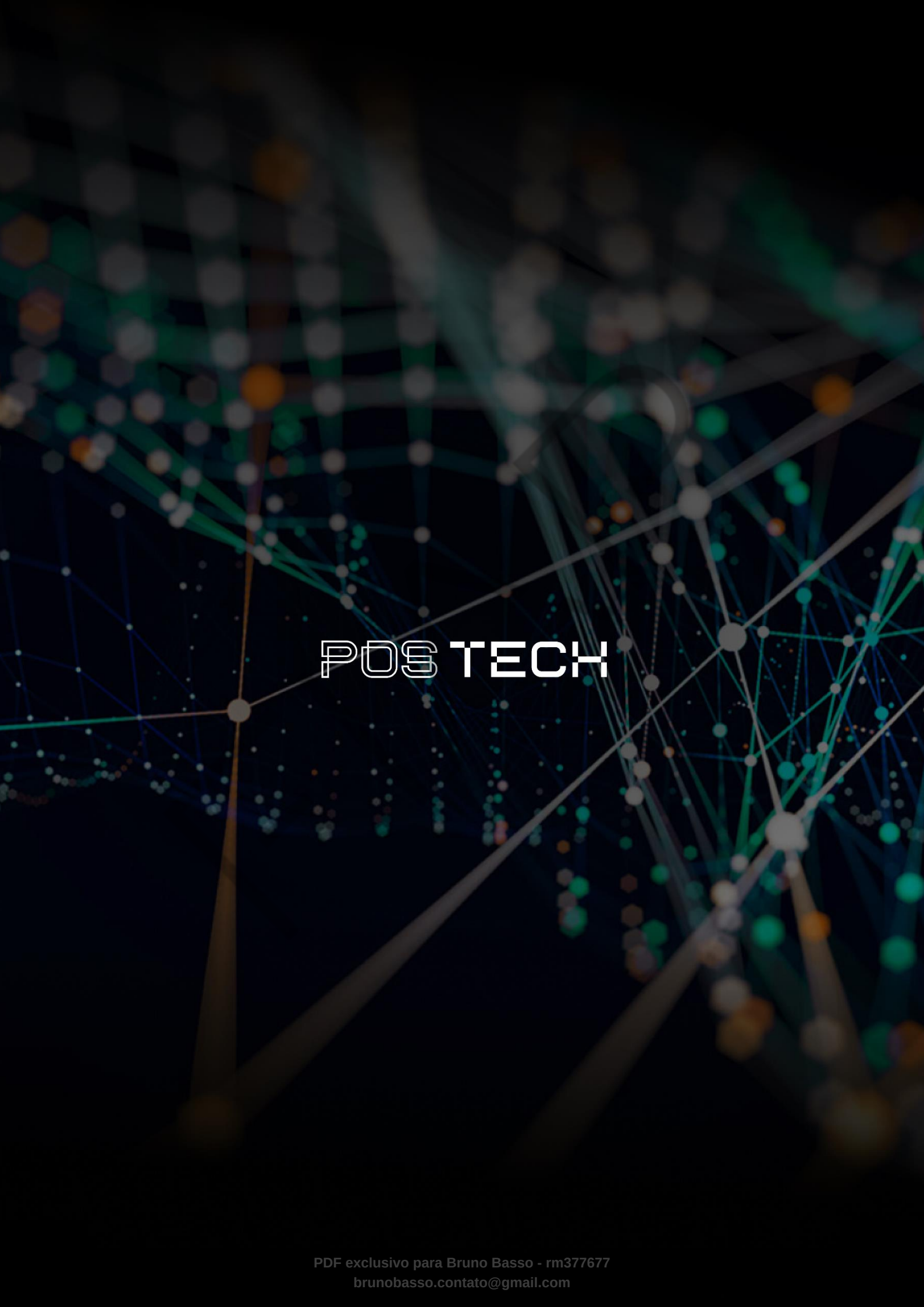
TERA BLOG. **Entenda o que é deploy de modelos em Machine Learning.** s.d.
Disponível em: <https://blog.somostera.com/data-science/deploy-o-que-e>. Acesso em:
28 jan. 2026.

EMSE

PALAVRAS-CHAVE

Deployment de modelos. MLFlow. Model Serving.

EMSE



POSTECH